TECNOLÓGICO NACIONAL DE MÉXICO

INSTITUTO TECNOLÓGICO DE VERACRUZ

Carrera:
Ing. en Sistemas Computacionales

Asignatura:
Administración de base de datos

Clave:
6J3

Grupo: A

Alumno:
Lopez Gonzalez Danna Nefertary

Número de control:
24020035

Práctica:
Crear un tablespaces en una Base de Datos que se aloje en un contenedor

Fecha de entrega: 20 de Octubre

Datos principales:

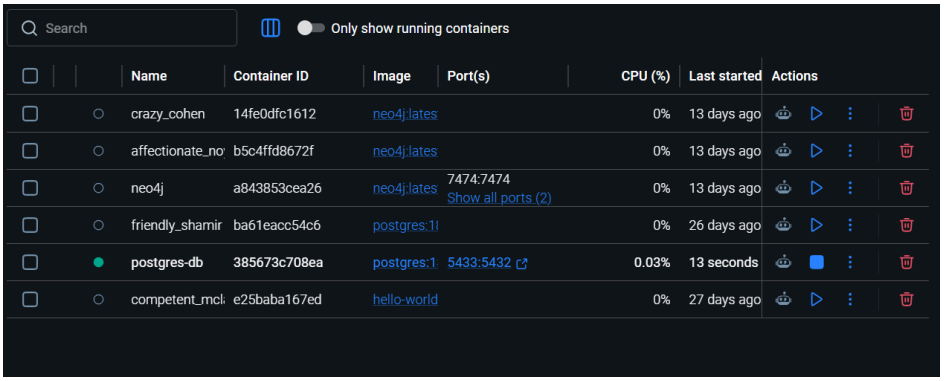
- Instalado y configurado Docker Desktop
- Configurado previamente nuestra base de datos dentro del Docker Desktop
- Utilización previamente de comandos con docker

Proceso:

- Abrimos la aplicación de Docker Desktop para después, verificar en la terminal PowerShell que contenedores se están ejecutando dentro de Docker

```
PS C:\Users\dNeTa> docker ps -a
CONTAINER ID   IMAGE          COMMAND                  CREATED       STATUS                     PORTS
 NAMES
14fe0dfc1612   neo4j:latest   "tini -g -- /startup…"   13 days ago   Exited (137) 12 days ago
 crazy_cohen
b5c4ffd8672f   neo4j:latest   "tini -g -- /startup…"   2 weeks ago   Exited (137) 12 days ago
 affectionate_noyce
a843853cea26   neo4j:latest   "tini -g -- /startup…"   2 weeks ago   Exited (137) 12 days ago
 neo4j
ba61eacc54c6   postgres:18    "docker-entrypoint.s…"   3 weeks ago   Exited (1) 3 weeks ago
 friendly_shamir
385673c708ea   postgres:18    "docker-entrypoint.s…"   3 weeks ago   Exited (255) 2 weeks ago   0.0.0.0:5433->5432/tcp
 postgres-db
e25baba167ed   hello-world    "/hello"                 3 weeks ago   Exited (0) 3 weeks ago
 competent_mclaren
```

- Como todos están en pausa, manualmente en la interfaz de la aplicación de Docker activamos nuestro contenedor donde se encuentra nuestra base de datos



- Ahora bien, ejecutamos este comando dentro de nuestra terminal *“docker exec -it postgres-db psql -U postgres”* quien manda al docker a entrar al contenedor de nombre Postgres-db, que abra una terminal interactiva y a su vez ejecute la consola de PostgreSQL conectándose con el usuario postgres. Si se ejecutó correctamente, el símbolo indicador sería “#” al final del nombre de usuario

```
PS C:\Users\dNeTa> docker exec -it postgres-db psql -U postgres
psql (18.6 (Debian 18.6-1.pgdg13+2))
Type "help" for help.
```

- En el siguiente paso, confirmaremos que base de datos tenemos creadas con el comando \l. En este caso tiene que estar creada la BD de nombre “Clase1”

```
postgres=# \l
postgres=#
```

```
                                                List of databases
   Name    |  Owner   | Encoding | Locale Provider |  Collate   |   Ctype    | Locale | ICU Rules |   Access privileges
-----------+----------+----------+-----------------+------------+------------+--------+-----------+-----------------------
--
 clase1    | postgres | UTF8     | libc            | en_US.utf8 | en_US.utf8 |        |           |
 postgres  | postgres | UTF8     | libc            | en_US.utf8 | en_US.utf8 |        |           |
 template0 | postgres | UTF8     | libc            | en_US.utf8 | en_US.utf8 |        |           | =c/postgres
 +
           |          |          |                 |            |            |        |           | postgres=CTc/postgre
s
 template1 | postgres | UTF8     | libc            | en_US.utf8 | en_US.utf8 |        |           | =c/postgres
 +
           |          |          |                 |            |            |        |           | postgres=CTc/postgre
s
(4 rows)
```

- Antes de crear un TABLESPACE, necesitamos una carpeta donde almacenar específicamente. En este caso, usamos dos comandos que nos muestra donde PostgreSQL guarda sus archivos de datos.

```
postgres=# SHOW data_directory;
        data_directory
------------------------------
 /var/lib/postgresql/18/docker
(1 row)

postgres=# SELECT current_setting('data_directory');
        current_setting
------------------------------
 /var/lib/postgresql/18/docker
(1 row)
```

- Para que no existan confusiones, crearé una nueva carpeta. Salimos de la opción de la terminal de PostgreSQL Y ejecutamos el siguiente comando lo crea dentro del contenedor con permisos de administrador del sistema

```
PS C:\Users\dNeTa> docker exec -u root postgres-db mkdir -p /tablespaces/ts_clase1
PS C:\Users\dNeTa>
```

- Siguiente, damos permisos al usuario del sistema operativo que ejecuta Postgres que será propietario de la carpeta

```
PS C:\Users\dNeTa> docker exec -u root postgres-db chown postgres:postgres /tablespaces/ts_clase1
PS C:\Users\dNeTa>
```

- Volvemos al comando que nos permite ejecutar la consola de PostgreSQL”*docker exec -it postgres-db psql -U postgres”* para ejecutar esta instrucción SQL que permite crear TABLESPACE

```
PS C:\Users\dNeTa> docker exec -it postgres-db psql -U postgres
psql (18.6 (Debian 18.6-1.pgdg13+2))
Type "help" for help.

postgres=# CREATE TABLESPACE ts_clase1
LOCATION '/tablespaces/ts_clase1';
CREATE TABLESPACE
postgres=#
```

- Mostramos si se creo nuestra TABLESPACE

```
postgres=# \db
              List of tablespaces
    Name    |  Owner   |        Location
------------+----------+------------------------
 pg_default | postgres |
 pg_global  | postgres |
 ts_clase1  | postgres | /tablespaces/ts_clase1
(3 rows)
```

Realizaremos una muestra final. Ahora podemos entrar directamente a la base de datos desde el contenedor. En el se ejecuta una tabla de prueba usando el TABLESPACE y finalmente comprobando donde se almacena.

1.-

```
PS C:\Users\dNeTa> docker exec -it postgres-db psql -U postgres -d clase1
psql (18.6 (Debian 18.6-1.pgdg13+2))
Type "help" for help.

clase1=#
```

2.-

```
clase1=# CREATE TABLE prueba_tablespace (
    id SERIAL PRIMARY KEY,
    descripcion VARCHAR(100)
) TABLESPACE ts_clase1;
CREATE TABLE
```

3.-

```
clase1=# SELECT tablename, tablespace
FROM pg_tables
WHERE tablename = 'prueba_tablespace';
     tablename     | tablespace
-------------------+------------
 prueba_tablespace | ts_clase1
(1 row)
```